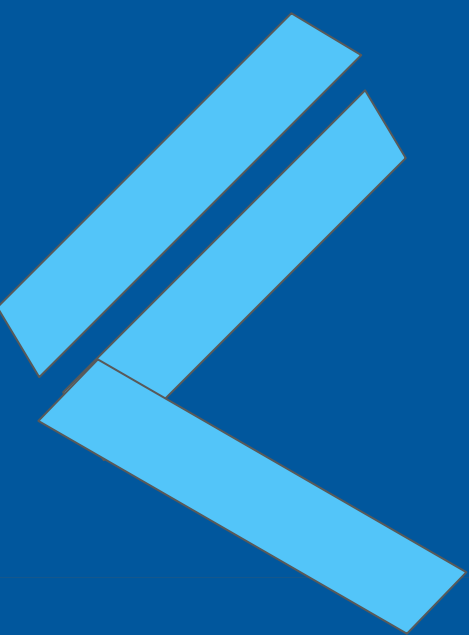




Desenvolvimento
Mobile
Aula 10

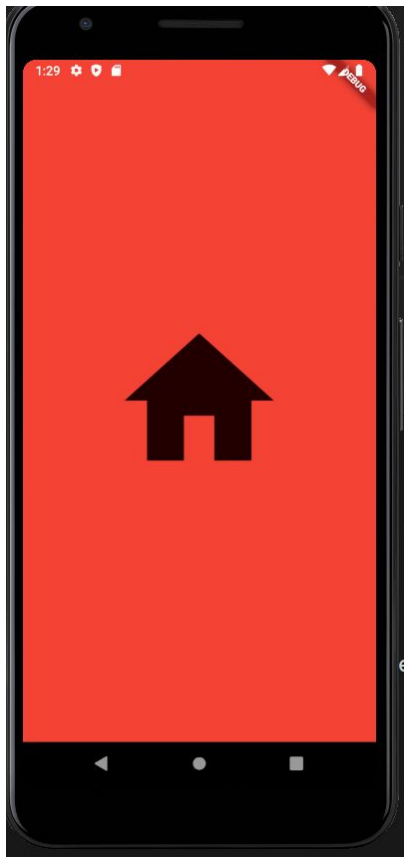
Prof. Me Daniel Vieira



Agenda

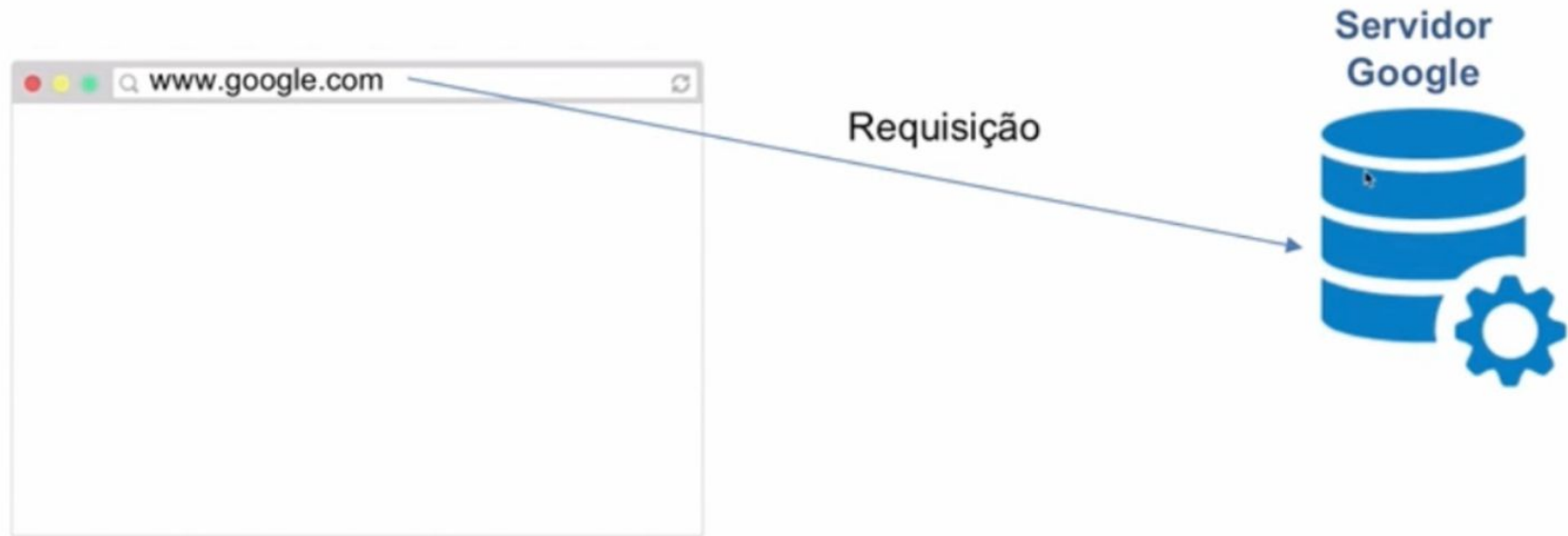
- 1- Requisições http
- 2 - Códigos de status
- 3-Apis
- 4 - Tipos de retorno de uma API
- 5 - Aplicativo consulta cep
- 6 - Criando animação na tela splash screen
- 7 - Personalizando o tema do aplicativo

Aplicativo para consulta de endereço pelo CEP



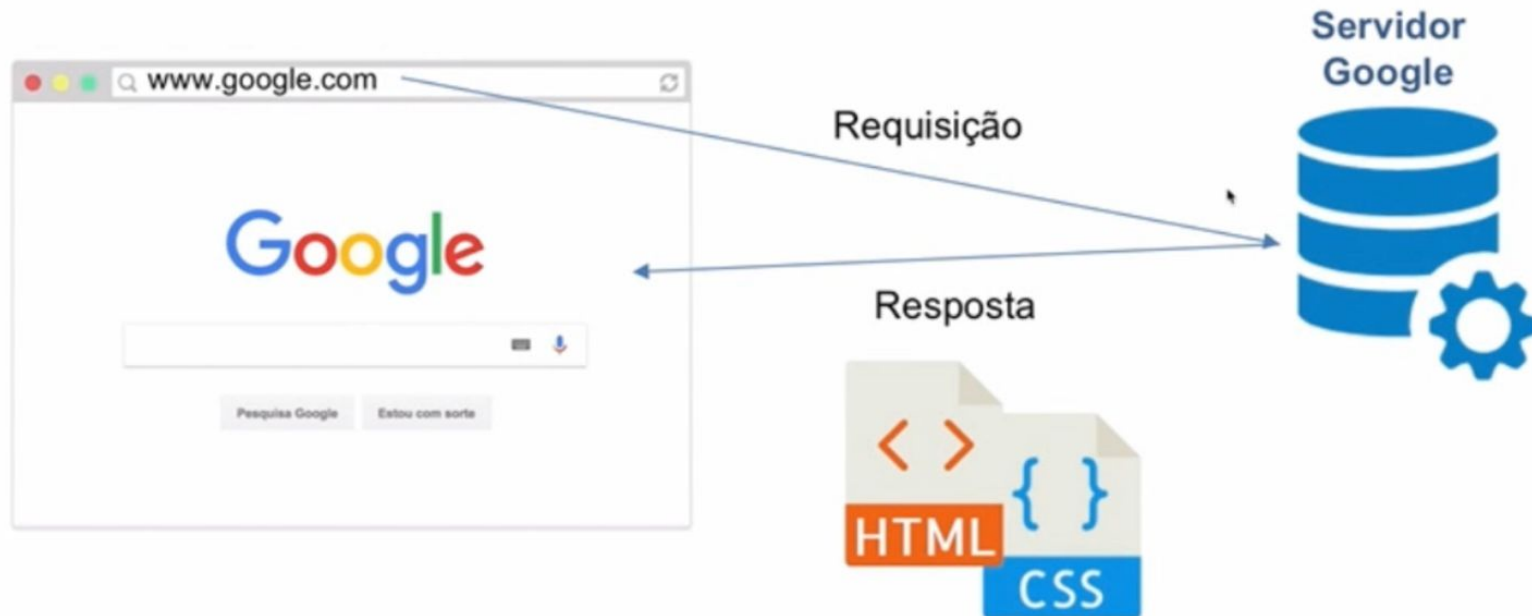
Requisições http

Como funciona a internet?



Requisições http

Como funciona a internet?



Requisições http

Http - Hypertext Transfer Protocol

Padronizou a comunicação entre navegadores e servidores

Tipos de requisições:

Get - Recupera dados no servidor

Post - Cria dado novo no servidor

Put - Atualiza dados no servidor

Delete - Deleta dados no servidor

Requisições http - Códigos de status

Códigos de status

404 - Not found

200 - Ok

501 - Bad Gateway

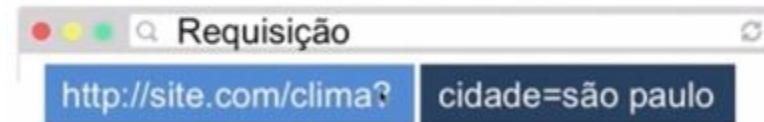
Utilizando API

API Application Programming Interface - Interface de programação de aplicações



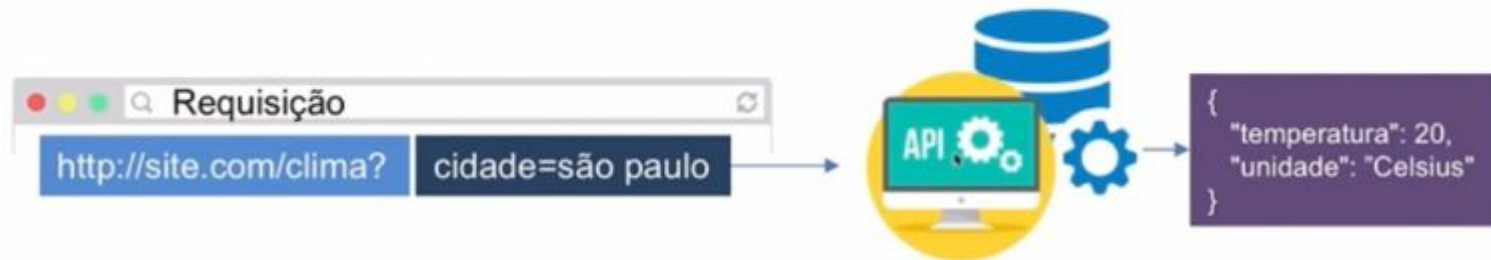
Utilizando API

API Application Programming Interface - Interface de programação de aplicações



Utilizando API

API Application Programming Interface - Interface de programação de aplicações



Tipos de retornos de API

O retorno precisa ser o mais genérico possível

XML (eXtensible Markup Language)

```
<resultado>
  <temperatura>20</temperatura>
  <unidade>Celsius</unidade>
</resultado>
```

```
{"cep":"13060360","address_type":"Rua","address_name":"Alzira Viegas Corrêa de Castro","address":"Rua Alzira Viegas Corrêa de Castro","state":"SP","district":"Residencial Parque da Fazenda","lat":"-22.92486","lng":"-47.12332","city":"Campinas","city_ibge":"3509502","ddd":"19"}
```

<https://cep.awesomeapi.com.br/13060360>

Tipos de retornos de API

O retorno precisa ser o mais genérico possível

XML (eXtensible Markup Language)

```
<resultado>  
  <temperatura>20</temperatura>  
  <unidade>Celsius</unidade>  
</resultado>
```

```
{"cep":"13060360","address_type":"Rua","address_name":"Alzira Viegas Corrêa de Castro","address":"Rua Alzira Viegas Corrêa de Castro","state":"SP","district":"Residencial Parque da  
Fazenda","lat":"-22.92486","lng":"-47.12332","city":"Campinas","city_ibge":"3509502","ddd":"19"}
```

<https://cep.awesomeapi.com.br/13060360>

Twitter API v2

Tweets

Bookmarks

- DELETE /2/users/:id/bookmarks/:tweet_id
- GET /2/users/:id/bookmarks
- POST /2/users/:id/bookmarks

Filtered stream

- GET /2/tweets/search/stream
- GET /2/tweets/search/stream/rules
- POST /2/tweets/search/stream/rules

Hide replies

- PUT /2/tweets/:id/hidden

Likes

Consumindo um api no Flutter

1º Necessário é no arquivo pubspec.yaml e adicionar a dependência no projeto

```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the Cupertino Icons font to your application.
35   # Use with the CupertinoIcons class for iOS style icons.
36   cupertino_icons: ^1.0.8
37   http: ^0.13.3
38   animated_splash_screen: ^1.3.0
39   page_transition: ^2.0.9
40 dev_dependencies:
41   flutter_test:
42     sdk: flutter
```

2º Importar o método http no arquivo necessário

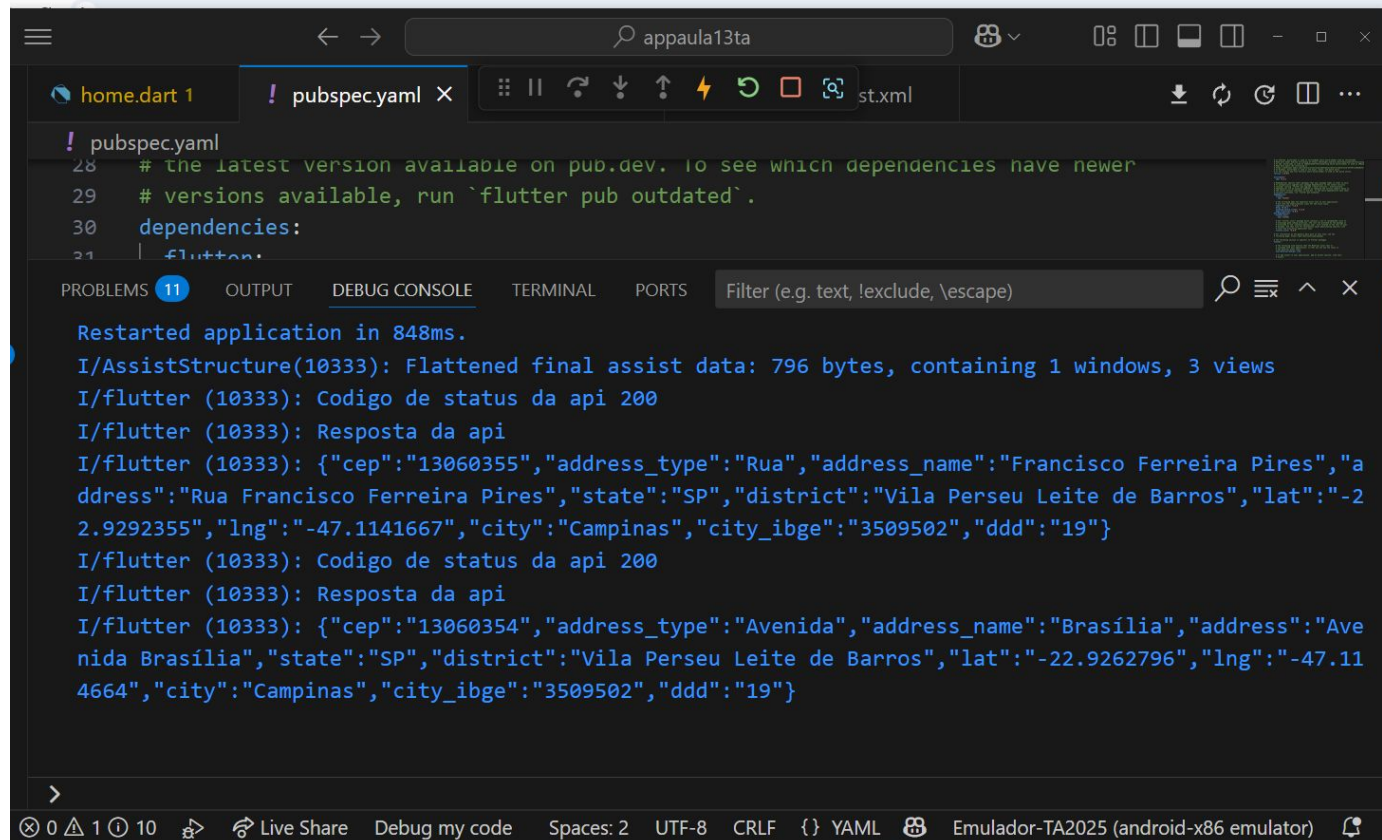
Consumindo um api no Flutter

1º Necessário é no arquivo pubspec.yaml e adicionar a dependência no projeto

```
# The following adds the Cupertino Icons font to your application.  
# Use with the CupertinoIcons class for iOS style icons.  
cupertino_icons: ^1.0.2  
http: ^0.13.3
```

2º Importar o método http no arquivo necessário

Consumindo um api no Flutter



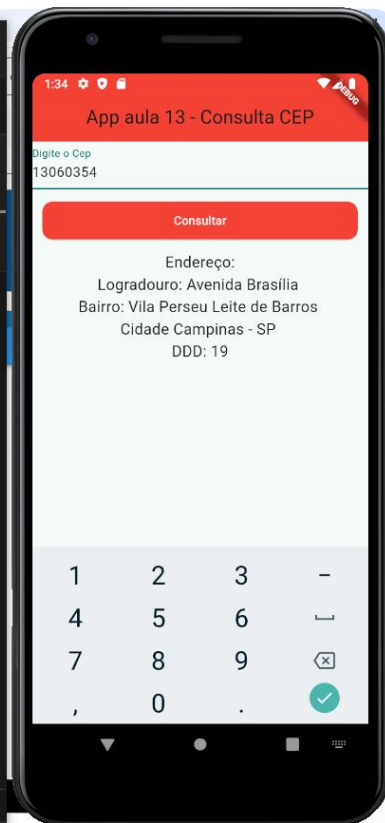
The screenshot shows an IDE window with a dark theme. The top toolbar includes icons for navigation, search, and window management. The active file is `pubspec.yaml`, which contains the following content:

```
pubspec.yaml
28 # the latest version available on pub.dev. To see which dependencies have newer
29 # versions available, run `flutter pub outdated`.
30 dependencies:
31   flutter:
```

The bottom panel displays the `DEBUG CONSOLE` with the following output:

```
Restarted application in 848ms.
I/AssistStructure(10333): Flattened final assist data: 796 bytes, containing 1 windows, 3 views
I/flutter (10333): Código de status da api 200
I/flutter (10333): Resposta da api
I/flutter (10333): {"cep":"13060355","address_type":"Rua","address_name":"Francisco Ferreira Pires","a
ddress":"Rua Francisco Ferreira Pires","state":"SP","district":"Vila Perseu Leite de Barros","lat":"-2
2.9292355","lng":"-47.1141667","city":"Campinas","city_ibge":"3509502","ddd":"19"}
I/flutter (10333): Código de status da api 200
I/flutter (10333): Resposta da api
I/flutter (10333): {"cep":"13060354","address_type":"Avenida","address_name":"Brasília","address":"Ave
nida Brasília","state":"SP","district":"Vila Perseu Leite de Barros","lat":"-22.9262796","lng":"-47.11
4664","city":"Campinas","city_ibge":"3509502","ddd":"19"}
```

The bottom status bar shows the following information: 0 errors, 1 warning, 10 info messages, Live Share, Debug my code, Spaces: 2, UTF-8, CRLF, {} YAML, Emulador-TA2025 (android-x86 emulator), and a notification icon.



Exercício

Criar um aplicativo para consumir a api cep awesome, onde o usuário deve digitar o numero do cep

The screenshot shows the AwesomeAPI documentation page. The header is green with the AwesomeAPI logo and a GitHub link. The left sidebar contains a navigation menu with 'Bem-vindo!', 'API de Cotações', and 'API CEP' (highlighted). The main content area shows the API endpoint `grpc://cep.awesomeapi.com.br` and a link to the GitHub repository. Below this is a 'Proto file' section. The 'API REST' section is expanded, showing a GET endpoint `https://cep.awesomeapi.com.br/format/cep` with the description 'Buscar um CEP'. It also includes a 'Parameters' table with columns for 'Path', 'format', and 'cep'.

AwesomeAPI the best for everyone [GitHub](#)

Bem-vindo!
API de Cotações
API CEP

`grpc://cep.awesomeapi.com.br`
<https://github.com/raniellyferreira/awesomeapi-cep/blob/master/proto/address.proto>

> Proto file

API REST

GET `https://cep.awesomeapi.com.br/format/cep`

Buscar um CEP

Retorna o endereço do CEP
Ex.: `https://cep.awesomeapi.com.br/json/05424020`

Parameters

Path		
format	string	Formato de resposta JSON (padrão), XML
cep	number	Apenas os números do CEP

Powered By GitBook

ON THIS PAGE
API gRPC
API REST
GET Buscar um CEP

Configuração para permitir que o apk acesse a internet

Para permitir que um aplicativo Flutter acesse a internet, você precisa definir permissões no arquivo `AndroidManifest.xml` para dispositivos Android e no arquivo `Info.plist` para dispositivos iOS.

Para dispositivos Android:

Você precisa adicionar a permissão `INTERNET` no arquivo `android/app/src/main/AndroidManifest.xml`. Certifique-se de que o seguinte código esteja presente dentro da tag `<manifest>`:

xml

 Copy code

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Configuração para permitir que o apk acesse a internet

android > app > src > main > AndroidManifest.xml

```
1  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2  <uses-permission android:name="android.permission.INTERNET"/>
3  <application
4      android:label="appaula13ta"
5      android:name="${applicationName}"
6      android:icon="@mipmap/ic_launcher">
7      <activity
8          android:name=".MainActivity"
9          android:exported="true"
10         android:launchMode="singleTop"
11         android:taskAffinity=""
12         android:theme="@style/LaunchTheme"
13         android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layoutDirection|fontScale|screenLayout|density|uiMode"
14         android:hardwareAccelerated="true"
15         android:windowSoftInputMode="adjustResize">
16         <!-- Specifies an Android theme to apply to this Activity as soon as
17             the Android process has started. This theme is visible to the user
18             while the Flutter UI initializes. After that, this theme continues
19             to determine the Window background behind the Flutter UI. -->
20         <meta-data
21             android:name="io.flutter.embedding.android.NormalTheme"
22             android:resource="@style/NormalTheme"
23         />
24     </activity>
25 </application>
26 </manifest>
```

Criando uma animação na tela splash

```
29     home: AnimatedSplashScreen(  
30         duration: 3000, // duração em segundos  
31         splash: Icons.home, // Poderia ser imagem, texto  
32         splashIconSize: 200, // aumenta o tamanho do icone  
33         splashTransition: SplashTransition.scaleTransition,  
34         pageTransitionType: PageTransitionType.leftToRight,  
35         nextScreen: Home(), // proxima tela  
36         backgroundColor: Colors.red,  
37     |  
38     |  
39     ) // AnimatedSplashScreen  
40  
41  
42 ); // MaterialApp  
43 }  
44 }
```

Personalizando o tema do aplicativo

```
12  @override
13  Widget build(BuildContext context) {
14    return MaterialApp(
15      theme: ThemeData(
16        useMaterial3: true,
17        colorScheme: ColorScheme.fromSeed(seedColor: Colors.red),
18        elevatedButtonTheme: ElevatedButtonThemeData(
19          style: ElevatedButton.styleFrom(
20            backgroundColor: Colors.red,
21            foregroundColor: Colors.white,
22            padding: EdgeInsets.symmetric(horizontal: 20, vertical: 12),
23            shape: RoundedRectangleBorder(
24              borderRadius: BorderRadius.circular(12)
25            ) // RoundedRectangleBorder
26          )
27        ) // ElevatedButtonThemeData
28      ), // ThemeData
```

Obrigado!

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95

